

Sprint 2D: Scenario Graph Completion

Renamed from “Duplicate Intelligence Removal”.

The old name framed this work as deleting code from Script Generation. That framing is backwards and dangerous — it invites us to delete inference before the graph can replace it, which just loses functionality.

The correct question is **NOT** “what code can we delete from Script Gen?”

The correct question is **“what information is still missing from the Scenario Graph?”**

Every PR must be **additive**: the graph gains a concrete capability. Deletion of the old inference code is the outcome of the graph becoming complete — never the objective, and never done first.

The Guiding Principle

Old (wrong) mental model:

Remove regex → Script Gen becomes dumber → hope the graph covers it

New (correct) mental model:

Make regex UNNECESSARY → Script Gen has nothing left to infer → deletion is trivial & safe

Each PR answers one question:

“What capability does the Scenario Graph gain?”

Not:

~~“What code did we delete?”~~

Why This Matters Beyond Script Generation

Once the Scenario Graph owns the full canonical **execution model** —

```
Scenario
  ↓
Execution Dataset    (2D.2)
  ↓
Executable Actions  (2D.3)  → step → action → target → value
  ↓
Executable Assertions (2D.4) → type → target → value
```

— then **every** downstream consumer reads the exact same model:

Script Generation
 Healing
 Replay
Self-Healing
 Coverage Gap Analysis
 Impact Analysis

That shared canonical execution model is a far bigger win than Script Generation itself.
 This is the highest-ROI change remaining in LevelUp.

The Migration Rule (read before every PR)

Never delete an inference path until the graph field that replaces it is populated, consumed, and proven by a test.

Additive PRs (2D.2 → 2D.4) come FIRST. They leave the old inference code untouched and simply prefer graph data when present. Only once the graph is complete do the deletion PRs (2D.5 → 2D.6) run — and by then they remove genuinely dead code.

Current State (Post-2D.1)

Sprint 2D.1 Complete

- Script Gen consumes `ScenarioSemantics` when available (`variableUnderTest` + `variation` + `expectedBehavior`)
- Stable `scenarioId`-based matching (no more fragile title matching)
- Credentials derived from semantics for graph-backed cases (bypasses `ScenarioIntelligence`)
- 4/4 unit tests passing

Reality check: the graph still describes steps as prose (`"Enter username"`), not executable actions. Script Gen still has to decide `fill()` vs `click()` , resolve the locator, resolve the value, and order the steps. So the inference code **cannot** be deleted yet — the graph doesn't carry enough information to replace it. That is exactly what 2D.3 and 2D.4 fix.

Implementation Order (revised — implement EXACTLY in this order)

- | | | |
|---|--|---|
| 2D.2 | Execution data | (graph owns resolved dataset values) |
|  | | |
| 2D.3 | Executable actions | (graph owns step  action  target  value) |
|  | | |
| 2D.4 | Executable assertions | (graph owns assertion  type  target  value) |
|  | | |
| 2D.5 | Delete duplicated inference | (regex/title/bucket detection  now dead) |
|  | | |
| 2D.6 | Delete <code>ScenarioIntelligence</code> | (module gone ; architecture contract enforces it) |

PR 2D.2 — Graph owns Execution Data ★★★★★

Capability the graph gains: every scenario node carries the concrete, resolved dataset values it will execute with — no downstream consumer ever re-derives them from step text.

Additive work:

- [] Script Gen reads username/password (and other fields) from `execution.resolvedDataset` on the `ScenarioNode`
- [] When `resolvedDataset` is present, it is the single source of truth for values
- [] Leave `extractCredentialsFromSteps()` / `looksLikeCredential()` in place as fallback for legacy cases — do NOT delete yet

Target Files:

- `src/script-gen/script-gen-engine.ts` (credential derivation reads execution first)

Test Coverage:

- [] Unit test: Script Gen reads `resolvedDataset.username` + `resolvedDataset.password` from graph
- [] Unit test: `resolvedDataset` present → step-text extraction is NOT invoked
- [] Unit test: no `resolvedDataset` → legacy fallback still works (no regression)

Definition of Done: Values in generated scripts come from the graph, not from parsing prose.

PR 2D.3 — Graph owns Executable Actions ★★★★★

Capability the graph gains: each scenario step becomes a structured, executable instruction instead of prose. This is the pivotal PR — it's what makes the login/checkout regex unnecessary.

The shape the graph must now carry:

```
{
  "stepId": "S3",
  "action": "fill",
  "target": "username",
  "value": "@dataset.username"
}
```

`action` ∈ { navigate, fill, click, select, check, upload, wait, verify }

`target` = a stable element identity (semantic key), NOT a raw locator string.

`value` = literal or a `@dataset.*` reference resolved from 2D.2.

Additive work:

- [] Extend `ScenarioNode` step schema with `action` / `target` / `value` (populated upstream in the graph builder)
- [] Script Gen gains a deterministic mapper:

```
ts
switch (step.action) {
  case 'fill': return `await ${loc(step.target)}.fill(${val(step.value)});`;
  case 'click': return `await ${loc(step.target)}.click();`;
  case 'select': ...
  case 'verify': ...
}
```

- [] When a step carries a structured `action`, the mapper is used and **all inference is skipped for that step**
- [] Steps WITHOUT structured actions still flow through the existing regex path (legacy fallback — untouched)

Explicitly NOT in this PR:

-  Do NOT delete login-triad regex, checkout regex, `inferFlowBucket()`, or field-detection regex. They remain as the fallback for un-migrated steps. They become dead only in 2D.5.

Target Files:

- Scenario Graph builder (adds action/target/value to steps) — upstream
- `src/script-gen/script-gen-engine.ts` (deterministic action mapper)

Test Coverage:

- [] Unit test: step `{action:'fill', target:'username', value:'@dataset.username'}` → `await ...user-name.fill(user.username)` with zero regex
- [] Unit test: step `{action:'click', target:'login_button'}` → `await ...click()`
- [] Unit test: legacy prose step (no `action`) → still handled by fallback (no regression)

Definition of Done: A fully-migrated scenario produces its script by pure `switch(action)` mapping, with the regex path never entered.

PR 2D.4 — Graph owns Executable Assertions ★★★★★

Capability the graph gains: expected outcomes become structured, executable assertions instead of

abstract prose like `"Login succeeds"`. Script Gen never invents an assertion again.

`expectedBehavior: "Login succeeds"` is still too abstract. The graph must carry:

```
{
  "assertions": [
    { "type": "url",    "value": "/inventory" },
    { "type": "visible", "target": "logout_button" }
  ]
}
```

`type ∈ { url, visible, hidden, text, value, enabled, disabled, count, error }`

Additive work:

- [] Extend `ScenarioNode` with a structured `assertions[]` array (populated upstream)
- [] Script Gen gains a deterministic assertion mapper:

```
ts
switch (a.type) {
  case 'url':    return `await expect(page).toHaveURL(${re(a.value)});`;
  case 'visible': return `await expect(${loc(a.target)}).toBeVisible();`;
  case 'text':   return `await expect(${loc(a.target)}).toContainText(${str(a.value)});`;
  ...
}
```

- [] When structured `assertions[]` exists, use them exclusively
- [] Leave title/URL/error-fragment inference in place as fallback for un-migrated nodes

Explicitly NOT in this PR:

-  Do NOT delete `buildAssertion()` inference, coverage-category regex, or the `/login/` URL heuristics.

Deletion happens in 2D.5.

Target Files:

- Scenario Graph builder (adds structured assertions) — upstream
- `src/script-gen/script-gen-engine.ts` (deterministic assertion mapper)

Test Coverage:

- [] Unit test: `{type: 'url', value: '/inventory'} → toHaveURL(/inventory/)`
- [] Unit test: `{type: 'visible', target: 'logout_button'} → toBeVisible()`
- [] Unit test: `{type: 'error', value: 'Invalid credentials'} → toContainText('Invalid credentials')`
- [] Unit test: node without `assertions[]` → legacy inference still runs (no regression)

Definition of Done: A migrated scenario's assertions come entirely from the graph; Script Gen guesses nothing.

PR 2D.5 — Delete Duplicated Inference ★★★★★

Now — and only now — deletion is safe, because 2D.2–2D.4 made every one of these paths unreachable

for graph-backed scenarios. This PR removes what is, by this point, dead code.

What to delete (verify each is dead via coverage first):

- [] Login-triad detection (L3395–3577 in `script-gen-engine.ts`)
- [] Checkout detection (L3610 in `script-gen-engine.ts`; L304 in `page-object-rewriter.ts`)
- [] `inferFlowBucket()` auth/checkout bucketing (L1880–1910)
- [] Field-grounding regex (username/password/click detection, L4094–4138)
- [] Assertion inference from title/URL (`buildAssertion()` heuristics, L4299–4305, L4395–4401)
- [] Coverage-category regex (L2259–2290)
- [] `extractCredentialsFromSteps()` / `looksLikeCredential()` (L2055–2085) — replaced by 2D.2
- [] Post-login assertion trigger (L5366–5369)

Precondition (must hold before merging):

- [] Prove via test/telemetry that graph-backed scenarios never enter these paths
- [] Any remaining callers are legacy-only and explicitly scoped for removal

Test Coverage:

- [] Regression suite: all migrated scenarios produce identical (or better) scripts with the code removed
- [] No test relies on the deleted inference for a graph-backed case

PR 2D.6 — Delete ScenarioIntelligence ★★★★★

The final step: the classification module is now entirely obsolete.

What to DELETE:

- [] `src/script-gen/scenario-intelligence/` (entire directory: `index.ts`, `detection.ts`, `types.ts`,

all 6 transformers)

- [] `this.scenario = new ScenarioIntelligence()` (script-gen-engine.ts:595)
- [] `import { ScenarioIntelligence } ...` (script-gen-engine.ts:44)
- [] All `this.scenario.classify()` calls and transformer references

Architecture contract (enforced by test):

- [] `grep -r "ScenarioIntelligence" src/ → 0`
- [] `grep -r "scenario.classify" src/ → 0`
- [] `grep -r "transformer" src/script-gen/ → 0` (except unrelated type imports)
- [] Script Gen reads ONLY from ScenarioNode fields — zero classification, zero inference

Test Coverage:

- [] Architecture contract test: `script-gen-engine.ts` imports no scenario-intelligence module
- [] Architecture contract test: no regex-based scenario detection remains in `src/script-gen/`



Supporting Cleanup (opportunistic, after 2D.6)

- [] Remove auth-specific regex from `page-crawler.ts` (L309, L363)
- [] Remove login-specific stopwords from credential validator (L2070)
- [] Review `workflow-mapper.ts` login page detection (L94–95)



Progress Tracking

PR	Capability the Graph Gains	Additive?	Status	Tests
2D.1	Semantics (variable/variation/behavior)	✓	✓ DONE	4
2D.2	Execution data (resolvedDataset)	✓ additive	☐ TODO	TBD
2D.3	Executable actions (step·action·target·value)	✓ additive	☐ TODO	TBD
2D.4	Executable assertions (type·target·value)	✓ additive	☐ TODO	TBD
2D.5	— (deletion of now-dead inference)	🗑 removal	☐ TODO	TBD
2D.6	— (deletion of ScenarioIntelligence)	🗑 removal	☐ TODO	TBD



Success Criteria (End of Sprint 2D)

The Litmus Test — remove the requirement text entirely:

```
const requirement = undefined; // ✗ NEVER READ

// Script Gen works using ONLY the canonical graph node:
const node = scenarioGraph.get(tc.scenarioId);
const script = generateScript({
  title:          node.title,
  objective:      node.objective,
  execution:      node.execution, // 2D.2 – resolved dataset values
  steps:         node.steps,     // 2D.3 – each: action · target · value
  assertions:    node.assertions, // 2D.4 – each: type · target · value
});
```

Verified when:

1. The graph carries execution data, executable actions, and executable assertions
2. Script Gen is a pure `switch(action) / switch(assertion.type)` adapter
3. Removing the requirement text changes nothing in the output
4. Zero fallback to ScenarioIntelligence; zero regex classification
5. `src/script-gen/scenario-intelligence/` is deleted and the architecture contract test enforces it

And the same canonical execution model is now consumable by Healing, Replay, Self-Healing, Coverage Gap, and Impact Analysis.

Next Step: Start **PR 2D.2 — Graph owns Execution Data** (additive; `execution.resolvedDataset`).